

CSCE 629 ANALYSIS OF ALGORITHMS · MODULE 10 · LECTURE NOTES

Dynamic Programming

What to do when the subproblems overlap.

MODULE OUTCOMES

- Recognise overlapping subproblems and optimal substructure.
- Design a DP by defining the subproblem, recurrence, and base cases.
- Convert between memoised recursion and bottom-up tabulation.
- Analyse DP complexity as states $\times$ transitions.
- Reduce space when only recent rows are needed.

1 The signal: overlapping subproblems

Module 02 established that divide and conquer works when subproblems are independent. When they are not — when the same subproblem arises repeatedly — naive recursion recomputes it exponentially often.

The canonical demonstration is Fibonacci. Computing $\text{fib}(n)$ recursively makes approximately ϕ^n calls, yet there are only $n+1$ distinct values to compute. Everything beyond those $n+1$ is pure recomputation.

DYNAMIC PROGRAMMING IN ONE SENTENCE

Compute each distinct subproblem once, store the answer, and look it up thereafter.

That is the whole idea. The name is historical and actively misleading — Bellman chose it partly because it sounded impressive to a sceptical funder — and the technique is better described as 'recursion with a table'.

1.1 The two requirements

- **Optimal substructure.** An optimal solution to the problem contains optimal solutions to its subproblems. Shared with greedy algorithms (Module 09).
- **Overlapping subproblems.** The same subproblem is required many times. This is what separates DP from divide and conquer, and what makes the table worth keeping.

Both are needed. Without optimal substructure the recurrence is invalid; without overlap the table is pointless overhead and plain recursion is better.

2 The design method

Four questions, answered in order. The code is mechanical once they are answered, and essentially every difficulty people have with dynamic programming is a difficulty with question 1.

2.1 What is the subproblem?

Write the definition in English, precisely, before touching code. Good definitions look like:

- $dp[i][w]$ = the maximum value obtainable using only the first i items with total weight at most w .
- $dp[i][j]$ = the length of the longest common subsequence of $a[0..i)$ and $b[0..j)$.
- $dp[mask][v]$ = the shortest route visiting exactly the cities in $mask$ and ending at v .

Note what these have in common: each fixes a prefix, a subset, or a range, and each is a complete specification you could hand to someone else. If you cannot state your subproblem in one sentence, you do not yet have one, and no amount of coding will rescue that.

2.2 What is the recurrence?

How does a subproblem's answer follow from smaller ones? This is normally a case analysis over the *last decision*: take the item or do not; match the characters or delete from one side; place the parenthesis here or there.

2.3 What are the base cases?

The smallest subproblems, answerable directly. Empty prefix, empty set, zero capacity. Getting these wrong produces answers that are subtly off by a constant, which is harder to debug than a crash.

2.4 In what order?

Every entry must be computed after everything it depends on. Top-down memoisation handles this automatically — the recursion enforces it — which is one reason to write that form first. Bottom-up requires you to work the order out, and getting it wrong gives silently wrong answers rather than errors. Floyd–Warshall requiring k outermost (Module 07) is exactly this constraint.

3 Complexity

$$\text{time} = (\text{number of states}) \times (\text{work per state})$$

Problem	States	Per state	Total
Fibonacci	$O(n)$	$O(1)$	$O(n)$
0/1 knapsack	$O(nW)$	$O(1)$	$O(nW)$
Edit distance	$O(mn)$	$O(1)$	$O(mn)$
Matrix chain	$O(n^2)$	$O(n)$ — try every split	$O(n^3)$
Held–Karp TSP	$O(2^n n)$	$O(n)$	$O(2^n n^2)$

Count the table entries, multiply by the cost of filling one. There is nothing more to the analysis, which makes DP unusually easy to cost before you implement it — a real advantage when deciding whether an approach is affordable.

PSEUDO-POLYNOMIAL

Knapsack's $O(nW)$ looks polynomial, and in a sense it is — but not in the length of the input.

W is a *numeric value*. Writing it down takes about $\log_2 W$ bits, so an input containing W has size proportional to $\log W$, and $O(nW)$ is **exponential in the input length**. Doubling the number of bits used to write W squares the running time.

Algorithms of this shape are called **pseudo-polynomial**. The distinction is why knapsack can be NP-hard (Module 12) while admitting an $O(nW)$ algorithm, and it is a genuine trap when estimating feasibility.

4 Two implementations

4.1 Top-down: memoised recursion

```
from functools import lru_cache

@lru_cache(maxsize=None)
def lcs(i, j):
    if i == 0 or j == 0:
        return 0
    if a[i-1] == b[j-1]:
        return 1 + lcs(i-1, j-1)
    return max(lcs(i-1, j), lcs(i, j-1))
```

This is literally the recurrence, with a cache decorator. The recursion enforces dependency order for free, and only subproblems actually reachable from the top are computed — which can be a large saving when the state space is sparse.

4.2 Bottom-up: tabulation

```
def lcs_table(a, b):
    dp = [[0] * (len(b)+1) for _ in range(len(a)+1)]
    for i in range(1, len(a)+1):
        for j in range(1, len(b)+1):
            if a[i-1] == b[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])
    return dp[len(a)][len(b)]
```

	Top-down	Bottom-up
Ease of writing	Easier — it is the recurrence	You must determine the fill order yourself
States computed	Only those reachable from the top	All of them, whether needed or not
Overhead	Call stack and cache lookups	None — plain array indexing
Space optimisation	Awkward	Natural — keep only the rows still needed
Deep state chains	Risk of stack overflow	No risk

Recommendation: write top-down first, always. It is closer to the recurrence you reasoned about, so there is less to get wrong. Convert to bottom-up only when you need the space reduction or the constant factor.

5 Space reduction

When a DP row depends only on the previous row, you need not keep the whole table.

```
def knapsack(items, W):
    dp = [0] * (W + 1)
    for value, weight in items:
        for w in range(W, weight - 1, -1): # DOWNWARD
            dp[w] = max(dp[w], dp[w - weight] + value)
    return dp[W]
```

THE LOOP DIRECTION IS THE ALGORITHM

Iterating w **downward** means $dp[w - \text{weight}]$ still holds the value from the *previous* item's row, so each item is used at most once. This is 0/1 knapsack.

Iterating w **upward** means $dp[w - \text{weight}]$ may already have been updated for the *current* item, so the item can be used again. This is **unbounded** knapsack.

Two problems, one loop, opposite directions. It is among the neatest facts in the subject, and it is also a trap: get the direction wrong and you solve the other problem silently, with no error and a plausible answer.

The same technique applies to edit distance and LCS (keep two rows, or one row plus a diagonal), reducing $O(mn)$ space to $O(\min(m, n))$. The cost is that you can no longer reconstruct the actual solution by backtracking through the table — only its value. Hirschberg's algorithm recovers the alignment in linear space by combining this with divide and conquer, which is a pleasing collision of Modules 02 and 10.

6 The canon

Problem	Subproblem	Complexity	Note
0/1 knapsack	$dp[i][w]$: first i items, capacity w	$O(nW)$	Pseudo-polynomial.
Longest common subsequence	$dp[i][j]$: prefixes of length i and j	$O(mn)$	The basis of diff.
Edit distance	$dp[i][j]$: prefixes	$O(mn)$	Spell checking, DNA alignment, fuzzy search.
Longest increasing subsequence	$dp[i]$: LIS ending at i	$O(n^2)$ or $O(n \log n)$	The faster version uses patience sorting and binary search.
Matrix chain multiplication	$dp[i][j]$: best cost for the range	$O(n^3)$	Range DP — the pattern for many parsing problems.
Coin change	$dp[\text{amount}]$	$O(n \cdot \text{amount})$	Solves what greedy could not (Module 09).
Floyd–Warshall	$dp[k][i][j]$	$O(V^3)$	You already wrote this in Module 07.
Held–Karp TSP	$dp[\text{mask}][\text{last}]$	$O(2^n n^2)$	Exponential, but vastly better than $O(n!)$. Module 12.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
MIT 6.006 — Dynamic Programming I–IV	Four lectures built around the 'five easy steps' framework, which is close to the four questions used here. The best structured introduction available.
Jeff Erickson, Algorithms — Dynamic Programming chapter	Outstanding on how to <i>find</i> the subproblem, with many worked examples and an honest account of the false starts.
Tim Roughgarden — Dynamic Programming (Part 3)	Knapsack, sequence alignment, and optimal BSTs at an unhurried pace.
CSES Problem Set — Dynamic Programming section	19 problems ordered by difficulty, auto-graded. The single best free source of DP practice.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement Fibonacci three ways: naive recursion, memoised, and bottom-up with $O(1)$ space. Measure all three and find the n at which naive recursion becomes unusable.
- 2 Implement 0/1 knapsack with the 1-D array. Then flip the loop direction and verify you have solved unbounded knapsack. Explain the mechanism in writing.
- 3 Implement edit distance and extend it to reconstruct the actual sequence of operations by backtracking through the table.
- 4 Implement LIS in $O(n^2)$, then in $O(n \log n)$ with patience sorting. Measure both at $n = 10^5$.
- 5 Implement matrix chain multiplication. Verify on a chain where the optimal order is dramatically better than left-to-right, and report the ratio.
- 6 Implement Held–Karp TSP over subsets. Find the largest n you can solve in one minute, and compare with the $n!$ brute force.
- 7 Take a problem you solved greedily in Module 09 and where greed failed (coin change with $\{1,3,4\}$ is a good choice). Solve it with DP and confirm the optimal answer.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What property distinguishes a dynamic programming problem from a divide and conquer one?
- 2 State the four design questions in order, and say which one causes most difficulty.
- 3 How do you compute the complexity of a DP?
- 4 What is pseudo-polynomial time, and why is $O(nW)$ knapsack not polynomial in the input size?
- 5 Give two reasons to prefer top-down, and two to prefer bottom-up.
- 6 In 1-D knapsack, what does the loop direction determine, and why?